Agricultural Automation System Lab Examination

Questions and Model Solutions

Complete all five phases. Write Phases 1–4 on paper and implement Phase 5 in Java. This document contains an examination paper followed by an instructor model solution. Alternative coherent designs are acceptable.

Part A Examination Questions

Project scenario

A farm relies on manual monitoring of soil moisture, irrigation, fertilizer application, and crop health. This causes delayed decisions, unnecessary water consumption, and difficulty responding to changing weather. Design an Agricultural Automation System that monitors farm conditions, supports irrigation decisions, and notifies farmers.

Consider Farmer, Farm Administrator, IoT Sensor, Weather Service, Drone System, and AI Analytics System as potential actors. Define your system boundary and identify the features implemented in your examination solution.

Instructions

- Use a console application with simulated sensor readings and weather data. Actual hardware, internet access, a database, and AI model training are not required.
- Keep class names, responsibilities, CRC cards, UML diagrams, and Java code consistent.
- State assumptions. The implementation may focus on irrigation; fertilizer, pest detection, drone analysis, and harvesting may remain outside its scope.

Phase 1 Requirement Analysis

Question 1 Analyze the requirements

- Describe the system background and three problems with manual farming.
- State one primary objective and four specific objectives.
- Identify four actors and explain their interactions. Distinguish external devices and services from internal software.
- Specify four functional requirements and two non-functional requirements. At least two functional requirements must concern the implemented irrigation workflow.

Phase 2 Object Identification

Question 2 Identify and classify objects

- Identify five entity classes and provide at least two attributes for each.
- Identify three control classes and explain their responsibilities.
- Identify three boundary classes and explain the external interaction each supports.
- Present class name, classification, and attributes or responsibilities in tables. Justify the classifications.

Phase 3 CRC Card Development

Question 3 Prepare six CRC cards

Select six classes involved in soil monitoring, weather retrieval, irrigation decisions, and farmer notification. Each card must include the class name, at least two responsibilities, and collaborating classes. Include at least one entity, one control, and one boundary class across the six cards.

Phase 4 UML Modeling

Question 4 Draw a class diagram and a sequence diagram

a. Draw a UML class diagram for the irrigation subsystem with at least eight classes or interfaces. Show important attributes and methods, visibility, relevant associations and multiplicities, and interface realization or inheritance. Include classes supporting the design patterns. Use aggregation or composition only where justified by ownership and lifecycle semantics.

b. Draw a sequence diagram: a farmer requests an irrigation check; the system reads soil moisture, obtains weather through an adapter, applies the selected strategy, updates the pump, and notifies the farmer. Show participating objects, ordered messages, and an alt fragment for irrigation required versus not required.

Phase 5 Java Implementation

Question 5 Implement and demonstrate all five patterns

Pattern	Required behavior
Singleton	FarmDataManager has controlled construction and one shared instance; store simulated farm information and verify identity.
Factory	SensorFactory creates moisture and temperature sensors through Sensor; handle unsupported types. A Simple Factory is acceptable; identify the variant.
Observer	AlertPublisher supports subscription, notification, and removal. Use FarmerNotifier and FarmDashboard.
Strategy	Switch between NormalIrrigationStrategy and WaterSavingStrategy at runtime.
Adapter	WeatherAPIAdapter implements WeatherService and translates an incompatible simulated external API.

Simplified examination rules

Normal strategy: irrigate when moisture is below 40% and rain is not expected. Water-saving strategy: irrigate when moisture is below 25% and rain is not expected. Otherwise the pump remains off. Reject moisture outside 0–100%. These are examination rules, not agricultural recommendations.

The controller must obtain weather through the adapter and publish each valid irrigation decision to registered observers.

Required Demonstrations and Submission

Test	Input or action	Expected behavior
1	30%, clear, normal	Pump on
2	30%, clear, water saving	Pump off
3	20%, rain, each strategy	Pump off
4	40%, clear, normal	Pump off
5	25%, clear, water saving	Pump off
6	−5% and 105% moisture	Reject reading; pump off
7	Publish with two subscribers	Both receive notification
8	Remove one subscriber and publish	Only remaining subscriber receives it
9	Request manager twice	Same instance
10	Create both sensor types; request unknown type	Correct types; unknown type rejected

Submit handwritten Phases 1–4, Java source files, compilation and execution instructions, and a test table with expected and actual outcomes.

Part B Model Solutions

The following is one consistent solution. The analysis includes the broader farm domain, while the UML and Java implementation focus on irrigation. The executable example uses nested classes in one file for convenient compilation; the same classes may be placed in separate files.

Phase 1 Requirement Analysis Solution

Background and problems

Manual farm monitoring produces intermittent readings, delayed irrigation decisions, and inconsistent communication with farmers. Three problems are: soil conditions may change between inspections; irrigation can continue when water is unnecessary; and weather changes may not reach the person controlling irrigation in time.

Objectives

Primary objective: develop an object-oriented farm platform that makes traceable irrigation decisions using simulated soil and weather information.

- Monitor soil moisture through a common sensor interface.
- Select irrigation behavior according to the active strategy.
- Use external weather information to prevent irrigation when rain is expected.
- Notify farmers and dashboards of decisions while allowing future sensor and strategy extensions.

Phase 1 Solution Continued

System boundary and actors

The software includes domain data, controllers, the console interface, and device/service adapters. Physical sensors and weather providers are outside that boundary. Sensor objects inside the software represent readings; they are not the external physical devices themselves.

Actor	Interaction
Farmer	Requests a check and receives the decision through the console.
Farm Administrator	Configures sensor selection and irrigation strategy.
IoT Sensor	Supplies environmental readings through a gateway or simulated Sensor implementation.
Weather Service	Supplies forecasts through the weather adapter.

Drone System and AI Analytics System are extension actors for crop-health analysis; they do not participate in the implemented irrigation use case.

Functional requirements

- FR1: Create moisture and temperature sensors through a factory.
- FR2: Validate moisture and decide irrigation using the selected strategy and adapted forecast.
- FR3: Set the pump state and publish each valid irrigation decision to registered observers.
- FR4: Maintain field information through a shared FarmDataManager and support observer registration and removal.

Non functional requirements

- Maintainability: new sensor and strategy implementations should use existing interfaces without modifying irrigation decision coordination.
- Reliability: invalid moisture must be rejected and leave the pump off; equal inputs and strategy must produce the same decision.

Assumptions and scope

One field is checked at a time; sensor and forecast inputs are deterministic simulations. The program is single-threaded, uses in-memory storage, and performs no physical actuation. The administrator and farmer interactions are represented by console setup and output. Authentication, persistent storage, fertilizer scheduling, pest detection, drone operation, and model training are outside the executable scope.

Phase 2 Object Identification Solution

Entity classes

Class	Attributes	Reason
Field	id, cropType	Identifies a managed growing area.
Crop	cropID, healthStatus	Stores crop information; extension.
SoilMoistureSensor	sensorID, value	Stores device identity and moisture; code simplifies identity away.
WeatherData	rainExpected, observationTime	Represents forecast information; code retains rainExpected only.
IrrigationPump	pumpID, on	Represents device identity and state; code retains on only.

Phase 2 Solution Continued

Analysis attributes describe the fuller domain model. The Java example intentionally retains only the attributes needed for the assessed use case; these simplifications do not change the class responsibilities.

Control classes

Class	Responsibility
IrrigationController	Validate readings, coordinate weather and strategy, set pump state, and publish decisions.
SensorFactory	Create supported sensor objects and reject unsupported requests; creation service classified as control here.
AlertPublisher	Coordinate subscriber registration, removal, and delivery.

Boundary classes

Class	Responsibility
AASExam	Console entry point representing farmer requests and administrator setup.
WeatherAPIAdapter	Translate external weather codes into WeatherData.
FarmerNotifier	Deliver decisions to the farmer through simulated console output.

FarmDashboard is an additional output boundary. In a hardware deployment, a dedicated sensor gateway would acquire readings; the Sensor implementations are deterministic test substitutes here. Entity, control, and boundary are analysis roles rather than mandatory Java superclass types.

Phase 3 CRC Card Solution

IrrigationController

Responsibilities: Validate moisture; obtain weather; apply strategy; coordinate pump and alerts.

Collaborators: Sensor, WeatherService, WeatherData, IrrigationStrategy, IrrigationPump, AlertPublisher

SoilMoistureSensor

Responsibilities: Retain a simulated moisture value; identify sensor type; provide a reading.

Collaborators: SensorFactory, IrrigationController

WeatherAPIAdapter

Responsibilities: Request an external code; validate and translate the code into forecast data.

Collaborators: WeatherService, ExternalWeatherAPI, WeatherData

IrrigationPump

Responsibilities: Maintain activation state; accept state updates; report current state.

Collaborators: IrrigationController

AlertPublisher

Responsibilities: Maintain subscribers; remove subscribers; distribute event messages.

Collaborators: AlertObserver, IrrigationController

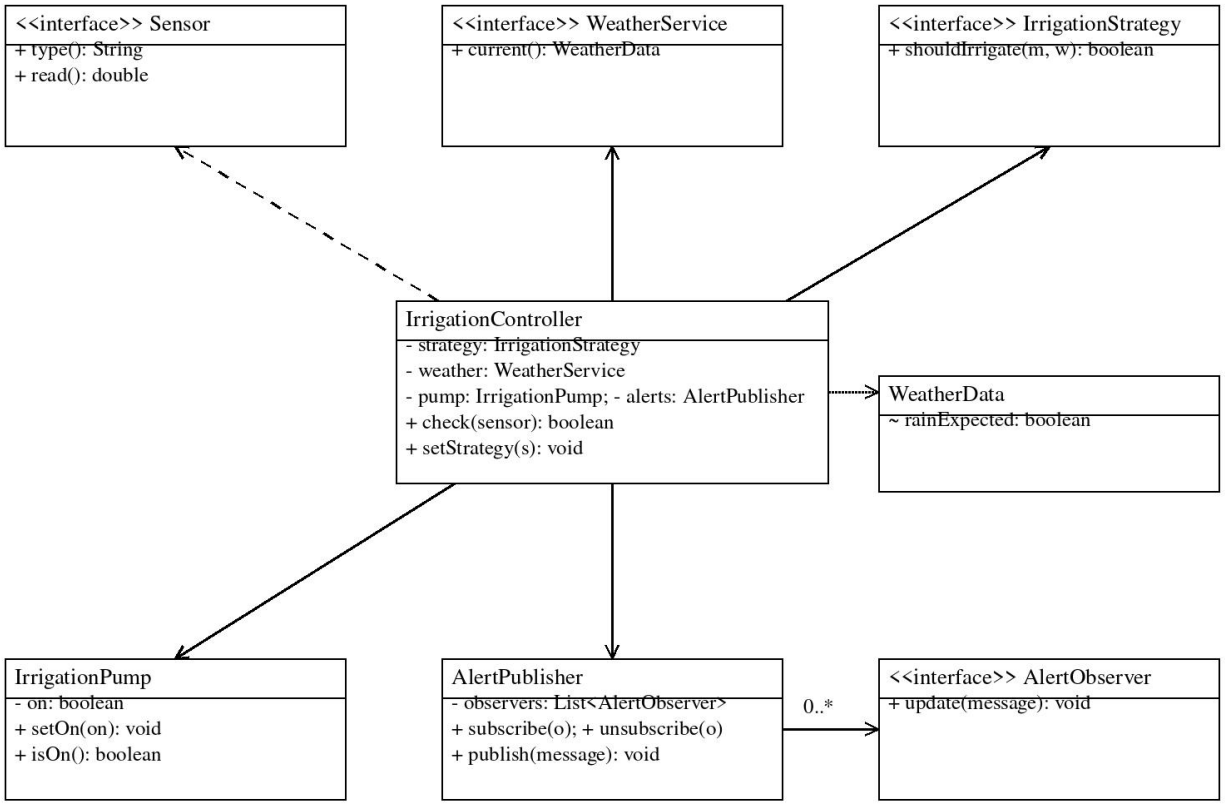
FarmerNotifier

Responsibilities: Receive alert messages; display them for the farmer; retain notification count for demonstration.

Collaborators: AlertObserver, AlertPublisher

Phase 4 UML Class Diagram Solution

The diagram below shows the core runtime collaboration. Dashed dependency arrows indicate method use; solid arrows indicate stored references. The accompanying relationship table covers the other pattern participants and their multiplicities.

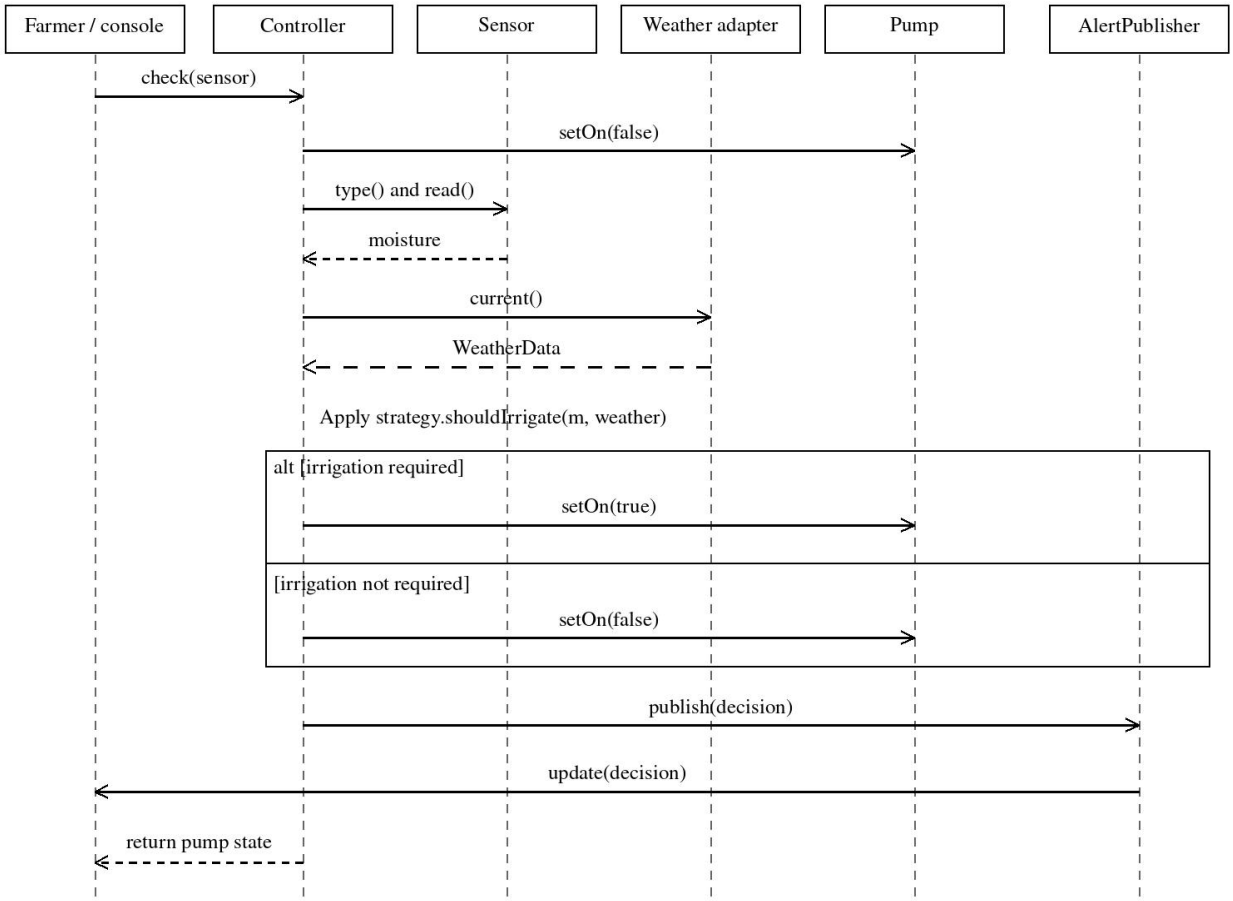


Controller holds one strategy, weather service, pump, and publisher.

Additional class or interface	Relationship
SoilMoistureSensor and TemperatureSensor	Each realizes Sensor. SensorFactory depends on both concrete classes and returns Sensor.
NormalIrrigationStrategy and WaterSavingStrategy	Each realizes IrrigationStrategy. The controller holds one selected strategy at a time.
WeatherAPIAdapter	Realizes WeatherService; holds one ExternalWeatherAPI and creates WeatherData.
FarmerNotifier and FarmDashboard	Each realizes AlertObserver. One AlertPublisher references zero or more observers.
FarmDataManager and Field	One manager references zero or more stored Field objects; fields are not lifecycle-owned parts.
AASExam	Depends on the factory, manager, controller, and concrete components for assembly and demonstration.

Phase 4 UML Sequence Diagram Solution

AASExam stands in for the farmer-facing console. Forecast retrieval is expanded in the notes below to show the adapter delegation. The alternative branches describe valid readings; invalid readings stop before forecast retrieval.



Adapter detail: WeatherAPIAdapter.current() calls ExternalWeatherAPI.fetchForecastCode(), maps CLEAR to rainExpected=false and RAIN to true, and returns WeatherData. Unknown codes raise an exception. Observer detail: publish() invokes update() on both FarmerNotifier and FarmDashboard; the diagram compresses the delivery to the farmer into one return message.

The controller calls the selected strategy before the alt fragment. Its pump reset occurs before validation so a bad reading does not leave a previously running pump on. All relationships are ordinary associations or dependencies; composition is unnecessary for the stated ownership model.

Phase 5 Java Solution

Save the complete listing as AASExam.java. It contains a Simple Factory, an eagerly initialized Singleton, Observer subscriptions, interchangeable Strategy objects, and a weather Adapter. Java visibility is intentionally package-level for helper classes and methods; interface implementations are public as required.

Compile: javac AASExam.java
Run: java AASExam

Verification note: a Java compiler was not available in the document-generation environment, so the listing has been reviewed but not compiled or executed here. The output below is expected behavior, not a recorded run. Run the included checks locally before using this as a grading reference.

The nested types keep the submission in one file. The Singleton restricts construction and shares a manager within this class-loader scope; it does not represent a production database or make mutable data operations thread-safe.

Java Solution Listing 1

```
import java.util.*;

public class AASExam {
  static final class Field {
    final String id, cropType;
    Field(String id, String cropType) {
      this.id = id; this.cropType = cropType;
    }
  }
  static final class FarmDataManager {
    private static final FarmDataManager INSTANCE = new FarmDataManager();
    private final Map<String, Field> fields = new LinkedHashMap<>();
    private FarmDataManager() { }
    static FarmDataManager getInstance() { return INSTANCE; }
    void save(Field field) { fields.put(field.id, field); }
    Field find(String id) { return fields.get(id); }
  }
  interface Sensor {
    String type();
    double read();
  }
  static final class SoilMoistureSensor implements Sensor {
    private final double value;
    SoilMoistureSensor(double value) { this.value = value; }
    public String type() { return "moisture"; }
    public double read() { return value; }
  }
  static final class TemperatureSensor implements Sensor {
    private final double value;
    TemperatureSensor(double value) { this.value = value; }
    public String type() { return "temperature"; }
    public double read() { return value; }
  }
  static final class SensorFactory {
    static Sensor create(String type, double value) {
      if ("moisture".equalsIgnoreCase(type))
        return new SoilMoistureSensor(value);
      if ("temperature".equalsIgnoreCase(type))
        return new TemperatureSensor(value);
      throw new IllegalArgumentException("Unsupported sensor: " + type);
    }
  }
  static final class WeatherData {
```

Java Solution Listing 2

```
  final boolean rainExpected;
  WeatherData(boolean rainExpected) { this.rainExpected = rainExpected; }
 }
 interface WeatherService { WeatherData current(); }
 static final class ExternalWeatherAPI {
  private final String forecast;
  ExternalWeatherAPI(String forecast) { this.forecast = forecast; }
  String fetchForecastCode() { return forecast; }
 }
 static final class WeatherAPIAdapter implements WeatherService {
  private final ExternalWeatherAPI api;
  WeatherAPIAdapter(ExternalWeatherAPI api) { this.api = api; }
  public WeatherData current() {
   String code = api.fetchForecastCode();
   if ("RAIN".equals(code)) return new WeatherData(true);
   if ("CLEAR".equals(code)) return new WeatherData(false);
   throw new IllegalArgumentException("Unknown weather code: " + code);
  }
 }
 interface IrrigationStrategy {
  boolean shouldIrrigate(double moisture, WeatherData weather);
 }
 static final class NormalIrrigationStrategy implements IrrigationStrategy {
  public boolean shouldIrrigate(double m, WeatherData w) {
   return m < 40 && !w.rainExpected;
  }
 }
 static final class WaterSavingStrategy implements IrrigationStrategy {
  public boolean shouldIrrigate(double m, WeatherData w) {
   return m < 25 && !w.rainExpected;
  }
 }
 interface AlertObserver { void update(String message); }
 static final class FarmerNotifier implements AlertObserver {
  int received;
  public void update(String message) {
   received++; System.out.println("Farmer: " + message);
  }
 }
 static final class FarmDashboard implements AlertObserver {
  int received;
  public void update(String message) {
   received++; System.out.println("Dashboard: " + message);
```

Java Solution Listing 3

```
  }
 }
 static final class AlertPublisher {
  private final List<AlertObserver> observers = new ArrayList<>();
  void subscribe(AlertObserver o) {
   if (!observers.contains(o)) observers.add(o);
  }
  void unsubscribe(AlertObserver o) { observers.remove(o); }
  void publish(String message) {
   for (AlertObserver o : new ArrayList<>(observers)) o.update(message);
  }
 }
 static final class IrrigationPump {
  private boolean on;
  void setOn(boolean on) { this.on = on; }
  boolean isOn() { return on; }
 }
 static final class IrrigationController {
  private IrrigationStrategy strategy;
  private final WeatherService weather;
  private final IrrigationPump pump;
  private final AlertPublisher alerts;
  IrrigationController(IrrigationStrategy s, WeatherService w,
                       IrrigationPump p, AlertPublisher a) {
   strategy = s; weather = w; pump = p; alerts = a;
  }
  void setStrategy(IrrigationStrategy s) { strategy = s; }
  boolean check(Sensor sensor) {
   // Fail safe: reset an already running pump before validation.
   pump.setOn(false);
   if (!"moisture".equals(sensor.type()))
    throw new IllegalArgumentException("Moisture sensor required");
   double m = sensor.read();
   if (!Double.isFinite(m) || m < 0 || m > 100) {
    alerts.publish("Invalid moisture; pump OFF");
    throw new IllegalArgumentException("Moisture must be 0 to 100");
   }
   WeatherData w = weather.current();
   boolean run = strategy.shouldIrrigate(m, w);
   pump.setOn(run);
   alerts.publish("Moisture=" + m + "; pump " + (run ? "ON" : "OFF"));
   return run;
  }
```

Java Solution Listing 4

```
 }
 static void expect(boolean condition, String name) {
  if (!condition) throw new AssertionError(name);
  System.out.println("PASS: " + name);
 }
 static void checkCase(IrrigationController c, IrrigationPump p,
                       double m, boolean expected, String label) {
  boolean result = c.check(SensorFactory.create("moisture", m));
  expect(result == expected && p.isOn() == expected, label);
 }
 public static void main(String[] args) {
  FarmDataManager data = FarmDataManager.getInstance();
  data.save(new Field("F1", "Rice"));
  expect(data == FarmDataManager.getInstance(), "singleton identity");
  expect("Rice".equals(data.find("F1").cropType), "stored field");
  expect(SensorFactory.create("moisture", 30) instanceof SoilMoistureSensor,
         "factory moisture");
  expect(SensorFactory.create("temperature", 28) instanceof TemperatureSensor,
         "factory temperature");
  try {
   SensorFactory.create("unknown", 0);
   throw new AssertionError("Unsupported type was accepted");
  } catch (IllegalArgumentException e) {
   System.out.println("PASS: unsupported sensor rejected");
  }
  AlertPublisher alerts = new AlertPublisher();
  FarmerNotifier farmer = new FarmerNotifier();
  FarmDashboard dashboard = new FarmDashboard();
  alerts.subscribe(farmer); alerts.subscribe(dashboard);
  alerts.publish("Weather alert");
  expect(farmer.received == 1 && dashboard.received == 1,
         "two observers notified");
  alerts.unsubscribe(dashboard);
  alerts.publish("Second alert");
  expect(farmer.received == 2 && dashboard.received == 1,
         "unsubscription");
  alerts.subscribe(dashboard);
  IrrigationPump pump = new IrrigationPump();
  WeatherService clear = new WeatherAPIAdapter(new ExternalWeatherAPI("CLEAR"));
  IrrigationController c = new IrrigationController(
     new NormalIrrigationStrategy(), clear, pump, alerts);
  checkCase(c, pump, 30, true, "normal 30 clear");
  c.setStrategy(new WaterSavingStrategy());
```

Java Solution Listing 5

```
    checkCase(c, pump, 30, false, "water saving 30 clear");
    checkCase(c, pump, 25, false, "water saving boundary 25");
    checkCase(c, pump, 20, true, "water saving 20 clear");
    c.setStrategy(new NormalIrrigationStrategy());
    checkCase(c, pump, 40, false, "normal boundary 40");
    WeatherService rain = new WeatherAPIAdapter(new ExternalWeatherAPI("RAIN"));
    IrrigationController wet = new IrrigationController(
        new NormalIrrigationStrategy(), rain, pump, alerts);
    checkCase(wet, pump, 20, false, "normal 20 rain");
    wet.setStrategy(new WaterSavingStrategy());
    checkCase(wet, pump, 20, false, "water saving 20 rain");
    for (double invalid : new double[] {-5, 105, Double.NaN}) {
      pump.setOn(true);
      try {
        c.check(SensorFactory.create("moisture", invalid));
        throw new AssertionError("Invalid reading was accepted");
      } catch (IllegalArgumentException e) {
        expect(!pump.isOn(), "invalid reading rejected with pump OFF");
      }
    }
    System.out.println("All checks completed.");
  }
}
```

Testing Solution and Interpretation

The main method runs deterministic checks and throws AssertionError if a checked result differs from its expectation. Every passing assertion prints PASS. Exceptions for unsupported sensors and invalid moisture are caught and checked. Ordinary Java assertions need not be enabled because expect() performs explicit checks.

Test	Expected result and code coverage
1	Pump on
2	Pump off
3	Pump off
4	Pump off
5	Pump off
6	Reject reading; pump off
7	Both receive notification
8	Only remaining subscriber receives it
9	Same instance
10	Correct types; unknown type rejected

Additional checks cover stored field retrieval, water-saving irrigation at 20% without rain, rejection of NaN, and rejection of bad readings while the pump was previously on. Both strategies are exercised with rain. The controller also rejects a temperature sensor passed to its moisture-check operation; add a dedicated test for this branch if extending the test suite.

Expected output excerpts

PASS: singleton identity
PASS: stored field
PASS: factory moisture
PASS: factory temperature
PASS: unsupported sensor rejected
PASS: two observers notified
PASS: unsubscription
PASS: normal 30 clear
PASS: water saving 30 clear
PASS: water saving boundary 25
PASS: normal boundary 40
All checks completed.

These excerpts omit interleaved farmer/dashboard messages and several additional passing checks. A valid decision sends a message such as Moisture=30.0; pump ON. An invalid reading produces Invalid moisture; pump OFF and an IllegalArgumentException. Students should capture actual output and compare it with the expected results.

Pattern explanation

The manager centralizes in-memory field access. The factory hides concrete sensor selection. The publisher separates decision production from subscriber delivery. Strategies isolate the two threshold policies and can be replaced at runtime. The adapter translates the external forecast representation into the interface expected by the controller.

Limitations and acceptable alternatives

This teaching example uses simulated inputs and simplified rules. Production use requires device integration, calibrated measurements, fault handling, security, persistence, and agronomic validation. Accept alternative designs when all five phases and requested patterns are consistent.